PBFT 톺아보기: 블록체인 합의 알고리즘의 뿌리

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 서울대학교 블록체인 학회 디사이퍼(Decipher) 스타일을 참고하여 작성되었습니다. PBFT(Practical Byzantine Fault Tolerance)의 원 논문을 기반으로 핵심 내용을 정리했습니다.

---

1. 들어가며

요즘 블록체인 합의 알고리즘을 공부하다 보면 PBFT라는 이름을 정말 많이 듣게 됩니다. Tendermint도 PBFT 기반이고, Cosmos 생태계도 그렇고, 심지어 이더리움 2.0의 Casper FFG도 BFT 계열이라고 합니다. 그래서 PBFT를 제대로 이해하면 요즘 나오는 합의 알고리즘들을 이해하는 데 도움이 될 것 같아서 원 논문을 읽어보게 되었습니다.

근데 막상 논문을 읽어보니까 생각보다 내용이 많고, 왜 이렇게 설계했는지에 대한 설명이 꽤 자세하게 나와 있었습니다. 그래서 이 글에서는 논문에 나온 내용을 바탕으로 PBFT가 어떤 문제를 풀려고 했고, 왜 이런 구조가 되었는지를 정리해 보려고 합니다.

---

2. 비잔틴 장애가 뭔데?

2.1 논문에서 말하는 비잔틴 장애

일단 PBFT가 뭘 해결하려는 건지부터 봐야 할 것 같습니다. 논문 서두를 보면 이런 내용이 나옵니다:

> "Malicious attacks and software errors are increasingly common. The growing reliance of industry and government on online information services makes malicious attacks more attractive and makes the consequences of successful attacks more serious."

악의적인 공격이랑 소프트웨어 오류가 점점 흔해지고 있다고 합니다. 그리고 산업이랑 정부가 온라인 서비스에 많이 의존하게 되면서 공격이 더 매력적인(?) 타겟이 되고 있고, 공격 성공 시 결과도 더 심각해진다고 합니다.

여기서 말하는 비잔틴 장애라는 게, 노드가 그냥 멈추는 게 아니라 **아무 행동이나 할 수 있는** 상황을 말합니다. 거짓말을 할 수도 있고, A한테는 이렇게 말하고 B한테는 저렇게 말할 수도 있고, 메시지를 받고도 안 받은 척 할 수도 있습니다. 단순히 서버가 죽는 Crash Fault보다 훨씬 다루기 어려운 상황인 거죠.

2.2 왜 기존 방법들은 안 됐나

논문을 더 읽어보면, PBFT 이전에도 비잔틴 장애를 다루려는 시도가 있었다고 합니다. 근데 문제가 있었습니다:

> "However, most earlier work either concerns techniques designed to demonstrate theoretical feasibility that are too inefficient to be used in practice, or assumes synchrony, i.e., relies on known bounds on message delays and process speeds."

기존 연구들은 두 가지 문제 중 하나가 있었다고 합니다. 첫째는 이론적으로 가능하다는 걸 보여주는 데 집중해서 실제로 쓰기엔 너무 느렸다는 것이고, 둘째는 **동기 시스템(synchronous system)**을 가정했다는 것입니다.

동기 시스템이 뭐냐면, 메시지가 언제까지는 반드시 도착한다는 걸 가정하는 겁니다. 예를 들어 "모든 메시지는 10초 안에 도착한다"라고 가정하는 식입니다. 근데 인터넷 같은 실제 환경에서는 이런 가정이 위험할 수 있습니다.

논문에서는 당시 가장 실용적이었던 Rampart랑 SecureRing이라는 시스템을 언급하면서 이렇게 말합니다:

> "The systems closest to ours, Rampart and SecureRing, were designed to be practical, but they rely on the synchrony assumption for correctness, which is dangerous in the presence of malicious attacks. An attacker may compromise the safety of a service by delaying non-faulty nodes or the communication between them until they are tagged as faulty and excluded from the replica group."

Rampart랑 SecureRing은 실용적으로 설계되긴 했는데, 정확성을 위해 동기 가정에 의존한다고 합니다. 이게 왜 위험하냐면, 공격자가 정상 노드들 사이의 통신을 지연시키면, 그 노드들이 장애 노드로 잘못 판정되어서 그룹에서 쫓겨날 수 있기 때문입니다.

이런 식의 공격은 실제로 노드를 해킹하는 것보다 훨씬 쉽습니다. 그냥 네트워크를 느리게 만들면 되니까요. 그래서 PBFT는 **Safety를 위해 동기 가정에 의존하지 않는** 방식으로 설계되었다고 합니다.

---

3. PBFT의 기본 구조

3.1 몇 개의 노드가 필요한가: 3f+1의 의미

PBFT 하면 항상 나오는 게 "3f+1" 입니다. f개의 비잔틴 노드를 허용하려면 전체 3f+1개의 노드가 필요하다는 건데, 논문에서는 왜 이 숫자가 나오는지 설명하고 있습니다:

> "The resiliency of our algorithm is optimal: 3f+1 is the minimum number of replicas that allow an asynchronous system to provide the safety and liveness properties when up to f replicas are faulty."

3f+1이 최적(optimal)이라고 합니다. 비동기 시스템에서 f개의 장애를 허용하면서 Safety와 Liveness를 보장하려면 최소한 이만큼이 필요하다는 것입니다.

왜 그런지에 대한 설명도 나옵니다:

> "This many replicas are needed because it must be possible to proceed after communicating with n-f replicas, since f replicas might be faulty and not responding. However, it is possible that the f replicas that did not respond are not faulty and, therefore, f of those that responded might be faulty. Even so, there must still be enough responses that those from non-faulty replicas outnumber those from faulty ones, i.e., n-2f > f. Therefore n > 3f."

좀 복잡한데, 정리하면 이렇습니다:

1. f개 노드가 장애일 수 있으니까, n-f개 노드랑만 통신해도 진행할 수 있어야 합니다.
2. 근데 응답 안 한 f개가 사실은 정상이고, 응답한 노드 중에 f개가 비잔틴일 수도 있습니다.
3. 그래도 정상 노드의 응답이 비잔틴 노드의 응답보다 많아야 합니다.
4. 즉, (n-f) - f > f 이어야 하고, 이걸 풀면 n > 3f가 됩니다.

그래서 최소 3f+1개가 필요한 것입니다.

3.2 Safety와 Liveness

논문에서는 PBFT가 보장하는 두 가지 속성을 설명합니다:

Safety (안전성): 정상 노드들은 절대로 서로 다른 값에 합의하지 않습니다. 논문에서는 이걸 linearizability라는 용어로 설명하는데, 쉽게 말하면 모든 연산이 마치 한 곳에서 순서대로 실행된 것처럼 보인다는 뜻입니다.

Liveness (생존성): 정상 노드가 보낸 요청은 결국 처리됩니다.

중요한 점은, 이 두 속성의 보장 조건이 다르다는 것입니다:

> "The algorithm does not rely on synchrony to provide safety. Therefore, it must rely on synchrony to provide liveness; otherwise it could be used to implement consensus in an asynchronous system, which is not possible."

Safety는 동기 가정 없이도 보장되지만, Liveness는 어느 정도 동기 가정이 필요하다고 합니다. 왜냐하면 완전한 비동기 시스템에서는 합의 자체가 불가능하다는 게 이미 증명되어 있기 때문입니다 (FLP impossibility라고 합니다).

이게 무슨 의미냐면, 악의적인 공격자가 네트워크를 마비시키면 시스템이 멈출 수는 있지만, 최소한 **잘못된 합의가 이루어지지는 않는다**는 것입니다. 차라리 멈추는 게 잘못된 결과를 내는 것보다 낫다는 설계 철학이 깔려 있는 것 같습니다.

---

4. 프로토콜 동작 방식

4.1 기본 구조: Primary와 Backup

PBFT는 **state machine replication** 방식을 사용합니다. 논문에서는 이렇게 설명합니다:

> "Our algorithm is a form of state machine replication: the service is modeled as a state machine that is replicated across different nodes in a distributed system."

서비스를 상태 기계(state machine)로 모델링하고, 이걸 여러 노드에 복제하는 방식입니다.

노드들은 **view**라는 단위로 운영됩니다:

> "The replicas move through a succession of configurations called views. In a view one replica is the primary and the others are backups. Views are numbered consecutively. The primary of a view is replica p such that p = v mod |R|, where v is the view number."

각 view에서 한 노드가 **primary**가 되고 나머지는 **backup**이 됩니다. Primary는 클라이언트 요청의 순서를 정하는 역할을 합니다. 어떤 노드가 primary가 되는지는 단순히 view 번호를 노드 수로 나눈 나머지로 결정됩니다.

4.2 3단계 프로토콜: Pre-prepare, Prepare, Commit

PBFT의 핵심은 3단계 프로토콜입니다. 논문에서는 각 단계의 역할을 이렇게 설명합니다:

> "The three phases are pre-prepare, prepare, and commit. The pre-prepare and prepare phases are used to totally order requests sent in the same view even when the primary, which proposes the ordering of requests, is faulty. The prepare and commit phases are used to ensure that requests that commit are totally ordered across views."

정리하면:

- **Pre-prepare + Prepare**: 같은 view 내에서 요청의 순서를 정합니다. Primary가 비잔틴이어도 동작합니다.
- **Prepare + Commit**: view가 바뀌어도 commit된 요청의 순서가 유지되도록 합니다.

Phase 1: Pre-prepare

Primary가 클라이언트 요청을 받으면, 시퀀스 번호를 붙여서 모든 backup에게 보냅니다:

```
⟨PRE-PREPARE, v, n, d⟩
```

- v: 현재 view 번호
- n: 시퀀스 번호
- d: 요청의 digest (해시값)

Backup은 이 메시지를 받으면 몇 가지를 검증합니다. 논문에서는:

> "A backup accepts a pre-prepare message provided: the signatures in the request and the pre-prepare message are correct and d is the digest for m; it is in view v; it has not accepted a pre-prepare message for view v and sequence number n containing a different digest; the sequence number in the pre-prepare message is between a low water mark, h, and a high water mark, H."

서명이 맞는지, view 번호가 맞는지, 같은 (v, n)에 대해 다른 요청을 받은 적이 없는지 등을 확인한다고 합니다.

Phase 2: Prepare

Pre-prepare를 받아들인 backup은 **PREPARE** 메시지를 다른 모든 노드에게 보냅니다:

```
⟨PREPARE, v, n, d, i⟩
```

그리고 2f개의 서로 다른 backup으로부터 같은 내용의 PREPARE를 수집하면 (자신의 Pre-prepare 포함 총 2f+1개), **prepared** 상태가 됩니다.

이 단계의 핵심은 논문에서 이렇게 설명됩니다:

> "The pre-prepare and prepare phases of the algorithm guarantee that non-faulty replicas agree on a total order for the requests within a view."

같은 view 내에서 정상 노드들이 요청의 순서에 동의하게 된다는 것입니다.

논문에서는 이게 왜 보장되는지도 설명합니다:

> "This is true because prepared(m,v,n,i) and |R|=3f+1 imply that at least f+1 non-faulty replicas have sent a pre-prepare or prepare for m in view v with sequence number n. Thus, for prepared(m',v,n,j) to be true at least one of these replicas needs to have sent two conflicting prepares (or pre-prepares if it is the primary for v), i.e., two prepares with the same view and sequence number and a different digest. But this is not possible because the replica is not faulty."

prepared 상태가 되려면 최소 f+1개의 정상 노드가 동의해야 하는데, 정상 노드는 같은 (v, n)에 대해 두 개의 다른 값에 동의할 수 없으니까, 서로 다른 두 요청이 같은 시퀀스 번호로 prepared 되는 일은 없다는 논리입니다.

Phase 3: Commit

Prepared 상태가 되면 **COMMIT** 메시지를 브로드캐스트합니다:

```
⟨COMMIT, v, n, d, i⟩
```

2f+1개의 COMMIT을 수집하면 **committed-local** 상태가 되고, 드디어 요청을 실행합니다.

왜 Commit 단계가 필요한지에 대해 논문에서는:

> "The commit phase ensures the following invariant: if committed-local(m,v,n,i) is true for some non-faulty i then committed(m,v,n) is true. This invariant and the view-change protocol described in Section 4.4 ensure that non-faulty replicas agree on the sequence numbers of requests that commit locally even if they commit in different views at each replica."

Commit 단계가 view가 바뀌어도 합의가 유지되도록 보장한다고 합니다.

4.3 왜 2단계가 아니라 3단계인가

이 부분이 처음에 좀 헷갈렸습니다. Prepare 단계에서 이미 2f+1개 동의를 받았으면 바로 실행하면 안 되나? 싶었거든요.

논문을 더 읽어보니까, 문제는 **view change** 상황에서 발생합니다. 내가 prepared 상태가 됐다고 해서 다른 노드들도 다 prepared 됐다는 보장이 없습니다. 만약 내가 prepared 되자마자 view change가 일어나면, 새로운 view의 primary는 내가 prepared 됐다는 사실을 모를 수 있습니다.

Commit 단계는 "나는 prepared 됐다"는 사실을 다른 노드들에게 알리는 역할을 합니다. 2f+1개의 Commit을 받았다는 건, 최소 f+1개의 정상 노드가 prepared 됐다는 뜻이고, view change가 일어나도 이 정보가 새 view로 전달될 수 있습니다.

---

5. View Change: 리더가 문제일 때

5.1 왜 View Change가 필요한가

지금까지 설명한 프로토콜은 primary가 정상이라고 가정했습니다. 근데 primary도 비잔틴일 수 있습니다. 악의적인 primary는:

- 특정 클라이언트의 요청을 무시할 수 있습니다
- 일부 backup에게만 pre-prepare를 보낼 수 있습니다
- 아예 아무것도 안 할 수 있습니다

이런 상황에서 시스템이 멈추지 않으려면 primary를 교체하는 메커니즘이 필요합니다. 이게 **view change**입니다.

5.2 View Change 동작 방식

논문에서는 view change가 어떻게 트리거되는지 설명합니다:

> "A backup is waiting for a request if it received a valid request and has not executed it. A backup starts a timer when it receives a request and the timer is not already running. It stops the timer when it is no longer waiting to execute the request, but restarts it if at that point it is waiting to execute some other request. If the timer of backup i expires in view v, the backup starts a view change to move the system to view v+1."

Backup은 요청을 받으면 타이머를 시작합니다. 일정 시간 안에 요청이 실행되지 않으면 타이머가 만료되고, view change를 시작합니다.

View change 메시지에는 중요한 정보가 담깁니다:

```
⟨VIEW-CHANGE, v+1, n, C, P, i⟩
```

- n: 마지막으로 실행한 시퀀스 번호
- C: 마지막 stable checkpoint의 증명
- P: prepared 됐지만 실행 안 된 요청들의 정보

P가 중요합니다. 새로운 primary가 이전 view에서 prepared 된 요청들을 알아야, 같은 순서로 다시 처리할 수 있기 때문입니다.

새 primary는 2f+1개의 VIEW-CHANGE 메시지를 수집하면 NEW-VIEW 메시지를 보내고, 정상 운영을 재개합니다.

5.3 View Change의 Safety 보장

논문에서는 view change가 safety를 어떻게 보장하는지 설명합니다:

> "A request commits locally at a non-faulty replica i with sequence number n in view v only if committed(m,v,n) is true. This means that there is a set R1 containing at least f+1 non-faulty replicas such that prepared(m,v,n,i) is true for every replica in the set."
>
> "Non-faulty replicas will not accept a pre-prepare for view v' > v without having received a new-view message for v'. But any correct new-view message for view v' > v contains correct view-change messages from every replica in a set R2 of 2f+1 replicas. Since there are 3f+1 replicas, R1 and R2 must intersect in at least one replica that is not faulty."

핵심은 **집합의 교집합**입니다:

1. Commit되려면 최소 f+1개 정상 노드에서 prepared 되어야 합니다 (집합 R1).
2. View change에 참여하는 2f+1개 노드 중 최소 f+1개는 정상입니다 (집합 R2).
3. 3f+1개 노드에서 R1과 R2는 최소 1개의 정상 노드를 공유합니다.
4. 이 정상 노드가 VIEW-CHANGE 메시지에 prepared 정보를 포함시킵니다.

그래서 이전 view에서 committed 된 요청은 새 view에서도 반드시 같은 순서로 처리됩니다.

---

6. 성능과 한계

6.1 통신 복잡도

PBFT의 가장 큰 한계는 통신 복잡도입니다. 정상 운영 시 메시지 복잡도는 **$O(n^2)$**입니다.

- Pre-prepare: 1 → n-1 = O(n)
- Prepare: n → n = $O(n^2)$
- Commit: n → n = $O(n^2)$

논문에서 실험 결과를 보면, 4개 노드(f=1)로 구성된 시스템에서 테스트했다고 합니다. 노드 수가 늘어나면 메시지 수가 급격히 증가하기 때문에, PBFT는 기본적으로 소규모 네트워크에 적합합니다.

6.2 View Change의 비용

View change는 더 비쌉니다. VIEW-CHANGE 메시지에 prepared 요청들의 정보가 포함되어야 하고, 이 정보에는 2f+1개의 PREPARE 메시지 증명이 필요합니다.

논문에서는 이 문제를 줄이기 위해 **checkpoint**를 사용한다고 합니다:

> "Generating these proofs after executing every operation would be expensive. Instead, they are generated periodically, when a request with a sequence number divisible by some constant (e.g., 100) is executed."

100개 요청마다 checkpoint를 생성해서, view change 시 전송해야 하는 데이터 양을 줄입니다.

6.3 논문에서 보고한 성능

논문에서는 BFS라는 비잔틴 장애 허용 NFS 파일 시스템을 구현해서 성능을 측정했습니다:

> "The results show that our service is only 3% slower than the standard NFS daemon in the Digital Unix kernel during normal-case operation."

표준 NFS보다 3%만 느리다고 합니다. 1999년 기준으로는 상당히 인상적인 결과였던 것 같습니다.

또한 기존 시스템과의 비교도 있습니다:

> "Even if we divide this latency by 1.7, the ratio of the SPECint92 ratings of the DEC 3000/400 and the SparcStation 10, our algorithm still reduces the latency to invoke the read-write and read-only 0/0 operations by factors of more than 10 and 20, respectively."

당시 가장 실용적이었던 Rampart보다 10~20배 빠르다고 합니다.

---

7. PBFT 이후의 발전

PBFT가 1999년에 나온 이후로 많은 후속 연구가 있었습니다. 특히 위에서 언급한 한계점들(고정 멤버십, 클라이언트-서버 구조, $O(n^2)$ 복잡도 등)을 해결하려는 시도들이 많았습니다. 몇 가지 대표적인 사례를 정리해 봤습니다.

7.1 Tendermint (2014)

Cosmos 생태계의 기반이 되는 합의 알고리즘입니다. PBFT를 블록체인에 맞게 수정했습니다.

주요 차이점:

- **라운드 기반 구조**: PBFT의 view change를 별도 프로토콜이 아니라 정상 운영의 일부로 통합했습니다. 매 블록마다 proposer가 바뀌는 구조여서 view change 비용이 크게 줄어듭니다.
- **Proposer 선출에 지분 가중치 적용**: PBFT에서는 단순히 `v mod n`으로 primary를 정했는데, Tendermint에서는 스테이킹 양에 비례해서 proposer가 될 확률이 높아집니다.

실제 사용 사례: Cosmos Hub, Osmosis, Terra(구), Cronos 등 Cosmos SDK 기반 체인들이 Tendermint를 사용합니다. Cosmos Hub의 경우 현재 180개 정도의 검증자가 운영되고 있는데, 검증자 집합 변경은 각 블록의 EndBlock에서 처리됩니다. Cosmos SDK의 `x/staking` 모듈이 이 로직을 담당하고 있고, 스테이킹/언스테이킹 요청이 들어오면 일정 기간(보통 21일)의 unbonding period 후에 검증자 집합에 반영됩니다.

7.2 HotStuff (2018)

Facebook(현 Meta)의 Libra/Diem 프로젝트를 위해 개발되었습니다. PBFT의 $O(n^2)$ 통신 복잡도 문제를 해결한 것으로 유명합니다.

핵심 아이디어 - Star Topology: PBFT에서는 모든 노드가 서로에게 메시지를 보내는 mesh 구조입니다. 그래서 Prepare 단계에서 n개 노드가 각각 n개 노드에게 메시지를 보내니까 $O(n^2)$이 됩니다. HotStuff는 이걸 리더 중심의 star 구조로 바꿨습니다. 노드들은 리더에게만 투표를 보내고, 리더가 이걸 모아서 **Quorum Certificate(QC)**를 만들어 브로드캐스트합니다.

Threshold Signature 활용: 2f+1개의 개별 서명을 하나의 서명으로 합칠 수 있는 threshold signature를 사용합니다. 이렇게 하면 QC의 크기가 노드 수와 관계없이 일정해져서 통신량이 O(n)으로 줄어듭니다.

View Change 단순화: PBFT에서 view change가 복잡하고 비용이 높았던 이유 중 하나가, 이전 view에서의 prepared 상태를 증명하기 위해 많은 메시지를 포함시켜야 했기 때문입니다. HotStuff는 3-chain commit rule을 도입해서 view change를 정상 프로토콜과 동일한 방식으로 처리합니다.

실제 사용 사례: 원래 Libra/Diem용으로 개발됐는데 프로젝트가 중단되면서, 그 기술이 Aptos의 DiemBFT로 이어졌습니다. Aptos 메인넷이 현재 HotStuff 변형을 사용하고 있습니다.

7.3 Casper FFG (2017)

이더리움 2.0(현재는 그냥 이더리움)의 finality gadget입니다. Vitalik Buterin과 Virgil Griffith가 설계했는데, PBFT에서 영감을 받았다고 합니다.

Finality Gadget이라는 접근: Casper FFG는 완전한 합의 프로토콜이 아니라 **finality gadget**입니다. 블록 생성은 다른 메커니즘(원래는 PoW, 지금은 LMD GHOST)이 담당하고, Casper FFG는 주기적으로 체크포인트에 대해 finality를 부여하는 역할만 합니다.

2/3 투표로 Justify → Finalize: 검증자들이 epoch(32 슬롯, 약 6.4분) 단위로 체크포인트에 투표합니다. 전체 스테이킹의 2/3 이상이 투표하면 해당 체크포인트가 **justified** 상태가 되고, justified된 체크포인트의 직접 자식도 justified되면 부모가 **finalized** 됩니다.

Slashing 조건: PBFT의 safety 증명이 "정상 노드는 conflicting vote를 안 한다"에 의존하는 것처럼, Casper FFG도 두 가지 slashing 조건을 정의합니다:

1. **Double voting**: 같은 epoch에 두 개의 다른 체크포인트에 투표
2. **Surround voting**: 이전 투표를 "감싸는" 투표를 함

이 조건을 어기면 스테이킹한 ETH가 슬래싱됩니다. 이게 PBFT의 "정상 노드" 가정을 경제적 인센티브로 대체한 것입니다.

실제 동작: 현재 이더리움 메인넷에서 약 100만 개의 검증자가 Casper FFG에 참여하고 있습니다. 각 epoch마다 검증자들이 attestation을 제출하고, 이 attestation에는 FFG vote(source와 target 체크포인트)가 포함됩니다. 보통 2 epoch(~13분) 후에 finality가 확정됩니다.

7.4 Sui의 Mysticeti (2024)

가장 최근 사례로, Sui 블록체인의 Mysticeti 합의가 있습니다. DAG 기반 BFT인데, PBFT의 3 message delay를 유지하면서 throughput을 크게 개선했습니다.

DAG 구조 활용: 기존 BFT가 순차적으로 블록을 쌓는 것과 달리, Mysticeti는 여러 검증자가 동시에 블록을 제안하고 이를 DAG(Directed Acyclic Graph)로 연결합니다. 이렇게 하면 네트워크 대역폭을 더 효율적으로 사용할 수 있습니다.

Uncertified DAG: 이전 DAG 기반 합의(예: Narwhal-Bullshark)는 각 블록에 대해 별도의 certification 과정이 필요했는데, 이게 3 message delay를 추가했습니다. Mysticeti는 certification 없이 DAG 자체의 구조를 이용해서 commit을 결정합니다.

실제 성능: Sui 메인넷에 2024년 8월 배포되었는데, 이전 Narwhal-Bullshark 대비 latency가 80% 감소했다고 합니다 (1900ms → 390ms). 이론적으로 BFT의 최소 latency인 3 message delay를 달성했다고 합니다.

Owned Object Fast Path: Sui의 특이한 점은 "owned object"(소유자가 명확한 객체)에 대한 트랜잭션은 합의 없이 처리할 수 있다는 것입니다. 예를 들어 단순 송금은 합의가 필요 없고, NFT 마켓플레이스처럼 여러 사용자가 같은 객체에 접근하는 경우에만 Mysticeti 합의를 거칩니다.

7.5 비교

특성	PBFT	Tendermint	HotStuff	Casper FFG	Mysticeti
통신 복잡도	$O(n^2)$	$O(n^2)$	O(n)	O(n)	O(n)
View Change	복잡/비쌈	라운드 통합	정상과 동일	없음(gadget)	DAG 구조
적합 규모	~20	~100	~100+	~100만*	~100
Finality	즉각	즉각	즉각	~13분	즉각

*Casper FFG는 검증자 수가 많지만, 실제로는 committee로 샘플링해서 처리

---

8. PBFT와 현대 블록체인의 차이점

논문을 읽다 보면 지금 우리가 알고 있는 블록체인 구조와 꽤 다르다는 걸 느끼게 됩니다. PBFT는 원래 블록체인을 위해 만들어진 게 아니라 **분산 시스템의 복제(replication)** 문제를 풀기 위해 만들어졌기 때문입니다. 몇 가지 차이점을 정리해 봤습니다.

8.1 클라이언트-서버 구조

PBFT 논문을 보면 **클라이언트(Client)**와 **레플리카(Replica)**가 명확히 분리되어 있습니다:

> "Clients issue requests to the replicated service to invoke operations and block waiting for a reply."

클라이언트는 요청을 보내고 응답을 기다리는 역할이고, 레플리카들이 합의를 수행합니다. 논문에서 구현한 BFS(Byzantine Fault-tolerant File System)를 보면 이 구조가 더 명확합니다:

> "Application processes run unmodified and interact with the mounted file system through the NFS client in the kernel. We rely on user level relay processes to mediate communication between the standard NFS client and the replicas."

일반 애플리케이션이 NFS 클라이언트를 통해 요청을 보내고, relay 프로세스가 이걸 레플리카들에게 전달하는 구조입니다.

반면 현대 블록체인에서는 이런 구분이 모호합니다. 누구나 트랜잭션을 보낼 수 있고, 검증자(validator) 노드들이 합의를 수행하긴 하지만, 일반 사용자도 풀노드를 돌릴 수 있고 상태를 직접 검증할 수 있습니다. "클라이언트"라는 개념이 딱히 없는 거죠.

8.2 Primary 주소를 알아야 하는 문제

논문에서는 클라이언트가 현재 primary가 누구인지 알아야 한다고 합니다:

> "A client sends a request to what it believes is the current primary using a point-to-point message."

클라이언트는 자기가 생각하는 현재 primary에게 **point-to-point 메시지**로 요청을 보냅니다. 그리고 view change가 일어나면 새 primary를 추적해야 합니다:

> "Each message sent by the replicas to the client includes the current view number, allowing the client to track the view and hence the current primary."

레플리카들이 보내는 응답에 현재 view 번호가 포함되어 있어서, 클라이언트가 현재 primary를 추적할 수 있다고 합니다.

이건 현대 블록체인과 상당히 다릅니다. 예를 들어 이더리움에서는 트랜잭션을 보낼 때 "현재 블록 제안자가 누구인지" 알 필요가 없습니다. 그냥 mempool에 트랜잭션을 던지면 되고, 어떤 검증자가 그걸 블록에 포함시킬지는 사용자가 신경 쓸 일이 아닙니다.

8.3 고정된 레플리카 집합

PBFT에서는 레플리카 집합이 고정되어 있습니다:

> "We denote the set of replicas by R and identify each replica using an integer in {0, ..., |R|-1}. For simplicity, we assume |R| = 3f+1 where f is the maximum number of replicas that may be faulty."

레플리카들은 0부터 |R|-1까지 번호가 매겨져 있고, 서로 누가 있는지 다 알고 있습니다. 새로운 노드가 중간에 참여하거나 빠지는 것에 대한 고려가 논문에는 없습니다.

현대 블록체인, 특히 퍼블릭 블록체인에서는 검증자 집합이 동적으로 변합니다. PoS 체인에서는 스테이킹하면 검증자가 될 수 있고, 언스테이킹하면 빠질 수 있습니다. 이런 **동적 멤버십(dynamic membership)** 문제는 PBFT 원 논문에서는 다루지 않습니다.

Cosmos SDK의 해결 방식: Tendermint 기반 체인들은 **epoch** 개념을 도입해서 이 문제를 해결합니다. Cosmos SDK의 `x/staking` 모듈을 보면, 검증자 집합 변경은 각 블록의 `EndBlock`에서 처리됩니다. 스테이킹 요청이 들어오면 바로 반영되는 게 아니라 unbonding period(보통 21일) 후에 반영됩니다. 이렇게 하면 합의 도중에 검증자 집합이 바뀌는 문제를 피할 수 있습니다.

Sui의 해결 방식: Sui는 epoch를 약 24시간 단위로 운영합니다. epoch 경계에서만 검증자 집합이 변경될 수 있고, epoch 내에서는 고정됩니다. Mysticeti 논문을 보면 이렇게 설명합니다: "quorum-based blockchains typically operate in epochs, allowing validators to join and leave the system at epoch boundaries."

이더리움의 해결 방식: 이더리움은 검증자 수가 100만 개가 넘어서 더 복잡합니다. 모든 검증자가 매번 투표하면 부하가 너무 크기 때문에, epoch(32 슬롯)마다 검증자들을 committee로 샘플링해서 일부만 투표에 참여시킵니다.

8.4 허가형(Permissioned) vs 비허가형(Permissionless)

위의 내용들을 종합하면, PBFT는 기본적으로 **허가형(Permissioned) 시스템**을 가정하고 있습니다. 누가 레플리카인지 미리 정해져 있고, 클라이언트도 레플리카들의 주소를 알고 있어야 합니다.

논문에서도 이런 가정이 드러납니다:

> "We use cryptographic techniques to prevent spoofing and replays and to detect corrupted messages. Our messages contain public-key signatures, message authentication codes, and message digests produced by collision-resistant hash functions. ... All replicas know the others' public keys to verify signatures."

모든 레플리카가 서로의 공개키를 알고 있다고 가정합니다. 이건 참여자가 미리 정해진 시스템에서나 가능한 얘기입니다.

비트코인이나 이더리움 같은 퍼블릭 블록체인은 **비허가형(Permissionless)** 시스템입니다. 아무나 참여할 수 있고, 누가 참여하고 있는지 미리 알 수 없습니다. 그래서 Nakamoto Consensus(PoW)나 PoS 같은 다른 방식이 필요했던 것입니다.

PoS 체인들의 절충안: 요즘 PoS 기반 체인들은 "누구나 검증자가 될 수 있다"는 점에서는 비허가형이지만, 실제 합의는 현재 활성화된 검증자들 사이에서 BFT 방식으로 이루어집니다. 즉, 참여 자체는 열려있지만 특정 시점의 합의 참여자는 정해져 있는 **반-허가형(semi-permissioned)** 구조라고 볼 수 있습니다.

예를 들어:

- **Cosmos Hub**: 상위 180개 검증자만 합의에 참여합니다. 누구나 검증자가 될 수 있지만, 스테이킹 양 기준 상위 180개에 들어야 실제로 블록 생성에 참여할 수 있습니다.
- **이더리움**: 32 ETH를 스테이킹하면 누구나 검증자가 될 수 있습니다. 검증자 수 제한은 없지만, 각 epoch에서 committee로 샘플링되어 실제 투표에 참여합니다.
- **Sui**: 매 epoch마다 스테이킹 양에 따라 검증자가 선정됩니다. 현재 약 100개 정도의 검증자가 운영되고 있습니다.

8.5 블록 개념의 부재

PBFT 논문에는 **블록(Block)**이라는 개념이 없습니다. 논문에서 합의하는 단위는 개별 **요청(Request)**입니다:

> "A client requests the execution of state machine operation o by sending a ⟨REQUEST, o, t, c⟩ message to the primary."

각 요청마다 시퀀스 번호가 붙고, 각 요청에 대해 3단계 프로토콜이 수행됩니다.

현대 블록체인에서는 여러 트랜잭션을 묶어서 **블록** 단위로 합의합니다. 이게 효율성 측면에서 훨씬 유리합니다. 논문에서도 이런 최적화를 언급하긴 합니다:

> "In this case, it buffers the request. Buffered requests are multicast later as a group to cut down on message traffic and CPU overheads under heavy load; this optimization is similar to a group commit in transactional systems."

요청을 버퍼링해서 그룹으로 처리하는 최적화가 있다고 하는데, 이게 나중에 블록체인에서 블록 단위 합의로 발전한 것 같습니다.

실제 블록체인에서의 batching:

- **이더리움**: 한 블록에 최대 ~1500개 정도의 트랜잭션이 들어갈 수 있습니다 (gas limit에 따라 다름). 12초마다 블록이 생성되고, 블록 단위로 합의가 이루어집니다.
- **Sui**: Mysticeti에서는 검증자들이 각자 블록을 제안하고, 이 블록들이 DAG로 연결됩니다. 각 블록에 여러 트랜잭션이 포함되고, DAG 구조 덕분에 여러 블록이 동시에 처리될 수 있습니다.
- **Cosmos 체인**: 블록당 트랜잭션 수는 체인마다 다르지만, 보통 몇 초에 한 블록씩 생성되고 블록 단위로 Tendermint 합의가 진행됩니다.

8.6 요약: PBFT에서 블록체인으로

구분	PBFT (1999)	현대 블록체인
구조	클라이언트-서버 분리	구분 모호, P2P
요청 전송	Primary에게 직접 전송	Mempool에 브로드캐스트
참여자	고정된 레플리카 집합	동적 멤버십
시스템 유형	허가형(Permissioned)	주로 비허가형
합의 단위	개별 요청	블록
Primary 정보	클라이언트가 추적 필요	사용자가 알 필요 없음

PBFT가 블록체인에 직접 쓰이기보다는, PBFT의 핵심 아이디어(3단계 합의, view change, 2f+1 정족수 등)를 가져와서 블록체인에 맞게 수정한 형태로 사용된다고 보는 게 맞는 것 같습니다.

---

9. 마치며

PBFT 논문을 읽으면서 느낀 건, 1999년에 이미 이렇게 체계적인 설계가 있었다는 게 좀 놀라웠습니다. 동시에 현대 블록체인과는 꽤 다른 가정들을 하고 있다는 것도 알게 됐습니다. 3f+1이 왜 필요한지, 3단계가 왜 필요한지, view change가 왜 복잡한지 등이 논문에 다 설명되어 있었습니다.

요즘 나오는 BFT 계열 합의 알고리즘들은 결국 PBFT의 한계를 극복하려는 시도들인 것 같습니다. 특히 $O(n^2)$ 통신 복잡도와 view change 비용 문제가 주요 개선 대상입니다.

블록체인 합의 알고리즘을 깊이 이해하고 싶다면, Castro와 Liskov의 원 논문을 한번 읽어보시는 걸 추천합니다. 14페이지 정도 되는 논문인데, 분산 시스템의 핵심 개념들이 잘 정리되어 있습니다.

References

1. Castro, M., & Liskov, B. (1999). *Practical Byzantine Fault Tolerance*. OSDI '99.
2. Lamport, L., Shostak, R., & Pease, M. (1982). *The Byzantine Generals Problem*. ACM TOPLAS.
3. Buchman, E. (2016). *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains*.
4. Yin, M., et al. (2019). *HotStuff: BFT Consensus with Linearity and Responsiveness*. PODC '19.